Some Notes

None of these will be counted against you in the current submission, but please try to respect the following from now on:

- Submit HTML or PDF
- Try to always show your results
- One subtask = one continuous job (save variables)
- It is good practice to interpret results
- Practice good code style (see Wickham et al. Ch. 4)

Solutions + Grading

- Example solution on GitHub
- Example solution is just an example
- Remember: submitting complete code = 3 pts
- Grades distributed this week via Moodle or Email
- Feedback as needed

What is Quanteda?

R package for quantitative text analysis, consists of:

- quanteda: The core functions
- quanteda.textmodels: To calculate and run ML models
- quanteda.textstats: Various text statistics formulas
- quanteda.textplots: Functions for text-related plots

Excellent tutorials on Quanteda Tutorials available

The Quanteda Workflow

Quanteda's workflow follows the Benoit text:

- Load texts: external functions (Sessions 11 & 12)
- Create a corpus: `corpus()`
- Defining documents and units of analysis
- Tokenize using various tokenizers: `tokens()`
- Create a document-feature matrix (DFM): `dfm()`
- Perform various analyses with subpackages

What Is a Corpus?

- Set of different texts we can analyse
- Can represent all (known) texts or a sample
- Can also be a subset
  - Usual sampling concerns apply
  - Think about unseen sampling biases!
- In Quanteda: Least abstracted form of text

What is a Document?

- One specific text we analyze
- One entry in a corpus
- We want documents to match units of analysis:
  - Full text (e.g. sentiment in parliamentary speeches)
  - Paragraph (e.g. in an interview)
  - Sentences (e.g. many known-category classifiers)
- Documents have associated variables (metadata / docvars)
- We often use docvars to "nest" documents somewhat

The Corpus Object in Quanteda

- Saves text split by documents
- Each entry consists of a text and docvars
- Functions similar to a data.frame
- Can be filtered with `corpus_subset()`
- Documents can be split with `corpus_reshape()`
- Access document variables with `docvars()`

ATTENTION: Corpus docvars are NOT fully tidyverse-compatible yet!

What is a Token?

- Tokens = information units of some sort
  - For classical (Western) QTA: Words and punctuation
  - For other languages: characters, groups of characters, ...
  - For other application: Morphemes, word parts, ...
- QTA almost always works with token-level data
- Tokenization is always pattern-based

Words and Tokens

- **Words:** Natural language units
- **Tokens:** linguistic groups
- Tokens can include non-words: symbols, punctuation, ...
- Tokens can be shorter than words: affices, clitics, etc.
- Tokens can be longer than words: n-grams, named entities, ...
- Tokens can be modified words: stems, lemmas, etc.
- **Types** are unique tokens

Tokenization

- Tokenizers are trained algorithms:
  - Simple tokenizers: use whitespace and punctuation
  - Can also be complex and trained (see Jurafsky & Martin)
- Quanteda uses a word-based tokenizer: `tokens()`
- Options to remove tokens (e.g. punctuation)
- We often edit tokens afterwards

Stopwords

- Stopwords = words that have grammatical functions: the, it, a
- Stopwords are usually the most common words in a text
- We often remove stopwords using pre-made lists
- In Quanteda: `tokens_remove(tokens, stopwords("en"))`
- Sometimes, removing stopwords is bad!

Stems and Lemmas

- Stems are words that have their suffixes removed
- Lemmas are the "basic" word form
- We use stems more often than lemmas
- Quanteda command: `tokens_wordstem()`

Word	**Stem**	**Lemma**
win	win	win
winning	win	win
won	*won*	*win*

Table 1: Table adapted from Stefan Müller's QTA class

The DFM

- A document-feature-matrix (DFM) is at the end of our pre-processing pipeline
- Shows how often each feature occurs in each document
- Feature = quantity of interest
- They are NOT tokens or words - why?

The DFM

- A document-feature-matrix (DFM) is at the end of our pre-processing pipeline
- Shows how often each feature occurs in each document
- Feature = quantity of interest
- They are NOT tokens or words - why?
- We change tokens from their textual form!

Working with a DFM

- We create the DFM with `dfm(tokens)`